

Writing Context-Free Grammars using NLTK

Complete Assignment Report (Parts A, B, C, and D)

Part A (5 Marks)

1. Visualizing the 5 Parse Trees Emitted by the NLTK Parser

The initial grammar parses the 11-word noun phrase 'a joke about the woman with the umbrella on the street' with start symbol NP using `nltk.parse.BottomUpChartParser`. Because the phrase contains a base NP ('a joke') followed by three Prepositional Phrases (PP1 = 'about the woman', PP2 = 'with the umbrella', PP3 = 'on the street'), the recursive production `NP -> NP PP` allows each PP to attach to different preceding NPs. This yields $C_3 = 5$ distinct parse trees:

- * **Tree 1:** [[[a joke] [about the woman]] [with the umbrella]] [on the street]] -- All 3 PPs attach to 'a joke'.
- * **Tree 2:** [[[a joke] [about [[the woman] [with the umbrella]]]] [on the street]] -- 'with the umbrella' modifies 'the woman'; 'on the street' modifies 'a joke...'
- * **Tree 3:** [[a joke] [about [[[the woman] [with the umbrella]] [on the street]]]] -- Both 'with the umbrella' and 'on the street' modify 'the woman'.
- * **Tree 4:** [[a joke] [about [[the woman] [with [[the umbrella] [on the street]]]]]] -- 'on the street' modifies 'the umbrella', which modifies 'the woman', which modifies 'a joke'.
- * **Tree 5:** [[[a joke] [about the woman]] [with [[the umbrella] [on the street]]]] -- 'about the woman' modifies 'a joke'; 'on the street' modifies 'the umbrella', which attaches to 'a joke about the woman'.

2. Top-Down Recursive Traversal (Non-Terminal Count & Word Positions)

We recurse through each emitted `nltk.Tree` top-down (pre-order) starting from the root node. In NLTK, every non-word node (both phrasal nodes [NP, PP] and pre-terminal POS nodes [Det, N, P]) is an instance of `nltk.tree.Tree`, while words are string leaves.

```
def recurse_tree_top_down(node, word_index=None, is_root=True):
    if word_index is None:
        word_index = [0]
    if not isinstance(node, Tree): # Base Case: leaf word printed during recursion
        idx = word_index[0]
        print(f'Word [{idx:2d}] (Position {idx + 1:2d}): {node}')
        word_index[0] += 1
        return 0, 0
    total_nt = 1
    phrasal_nt = 1 if node.height() > 2 else 0
    for child in node: # Top-Down recursion left-to-right
        c_tot, c_phr = recurse_tree_top_down(child, word_index, is_root=False)
        total_nt += c_tot; phrasal_nt += c_phr
    if is_root:
        print(f'Number of non-terminal nodes: {total_nt}')
    return total_nt, phrasal_nt
```

Output across all 5 parse trees (identical for Trees 1 to 5 because every tree applies 4 Det N rules, 3 P NP rules, 3 NP PP rules, and 11 POS rules):

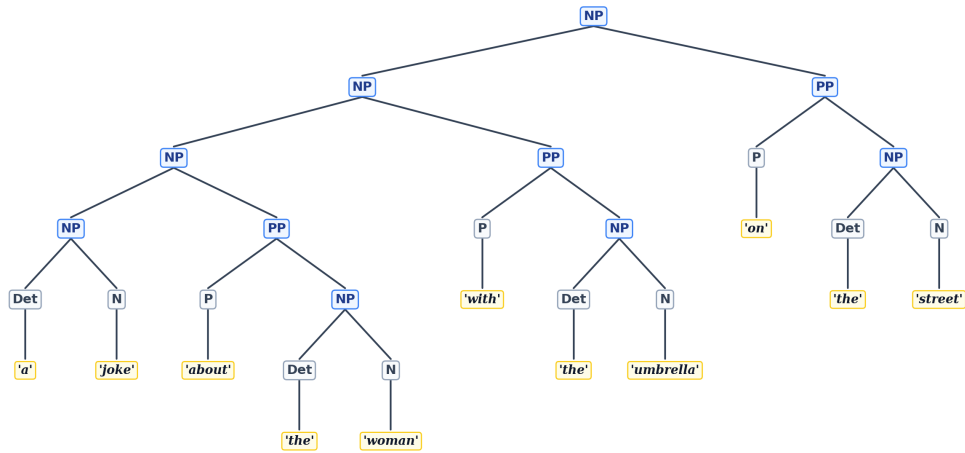
1. Number of non-terminal nodes (nodes which aren't words): 21
(Breakdown: 10 phrasal nodes [7 NP + 3 PP] + 11 POS-tag nodes [4 Det + 4 N + 3 P] = 21)
2. Words with numerical index indicating position in the sentence:

Index 0 (Pos 1): a	Index 6 (Pos 7): the
Index 1 (Pos 2): joke	Index 7 (Pos 8): umbrella
Index 2 (Pos 3): about	Index 8 (Pos 9): on
Index 3 (Pos 4): the	Index 9 (Pos 10): the
Index 4 (Pos 5): woman	Index 10 (Pos 11): street
Index 5 (Pos 6): with	

Part A.1 Visualizations: Emitted Parse Trees 1, 2, and 3

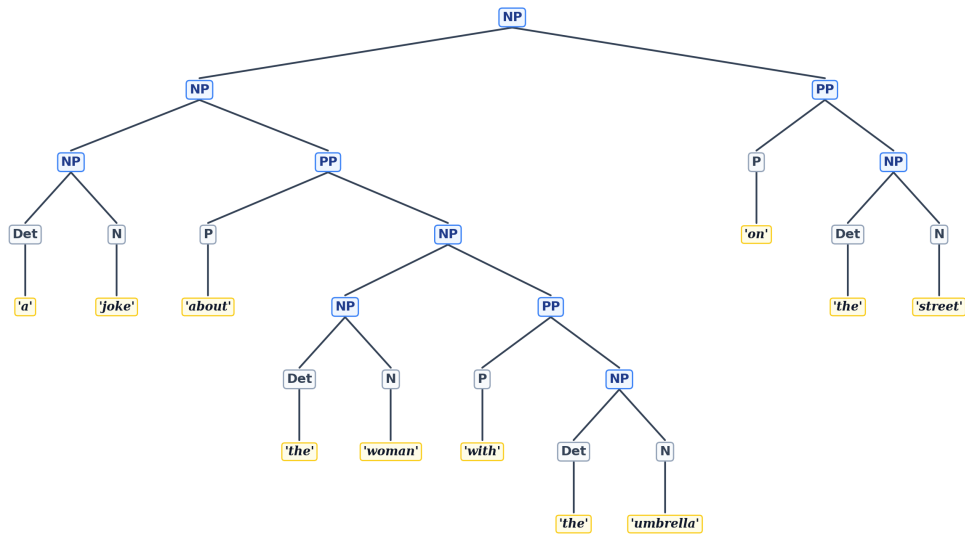
Part A — Parse Tree 1 of 5

PP1 ('about the woman'), PP2 ('with the umbrella'), and PP3 ('on the street') all attach to 'a joke'



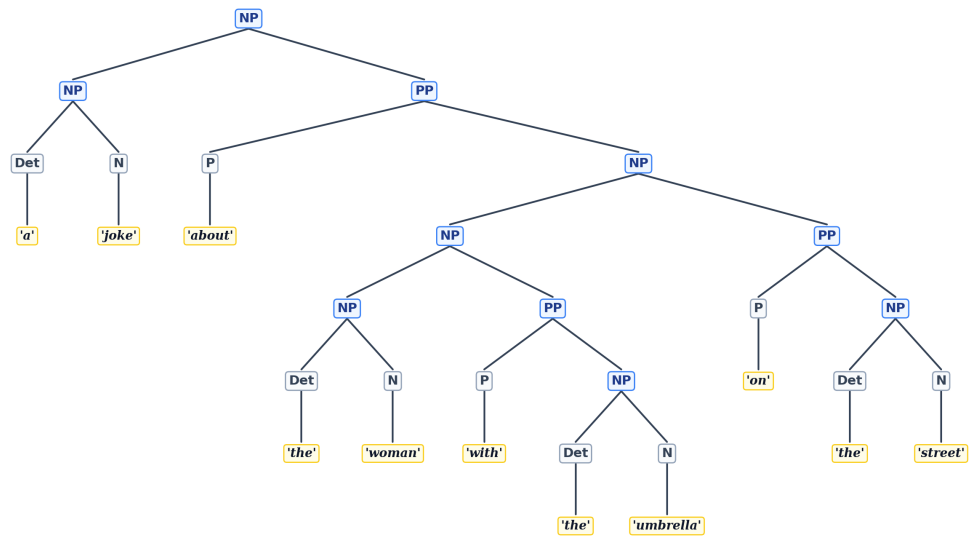
Part A — Parse Tree 2 of 5

PP2 ('with the umbrella') attaches to 'the woman'; PP3 ('on the street') attaches to 'a joke...'



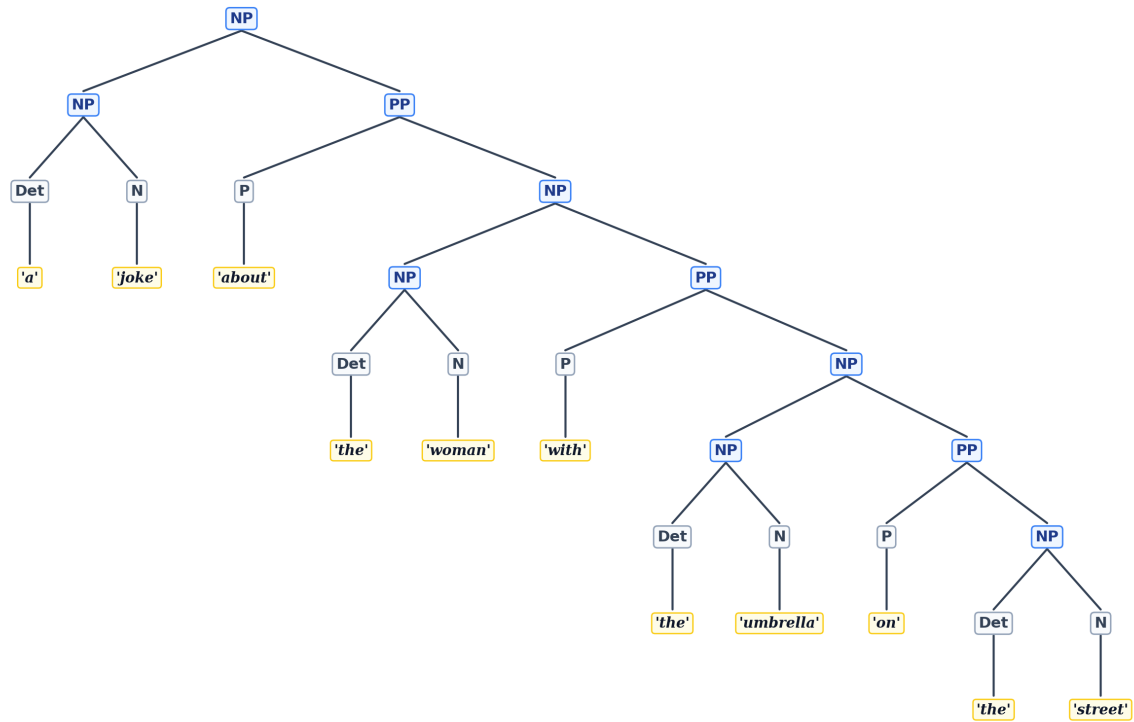
Part A — Parse Tree 3 of 5

PP2 ('with the umbrella') and PP3 ('on the street') both attach to 'the woman'

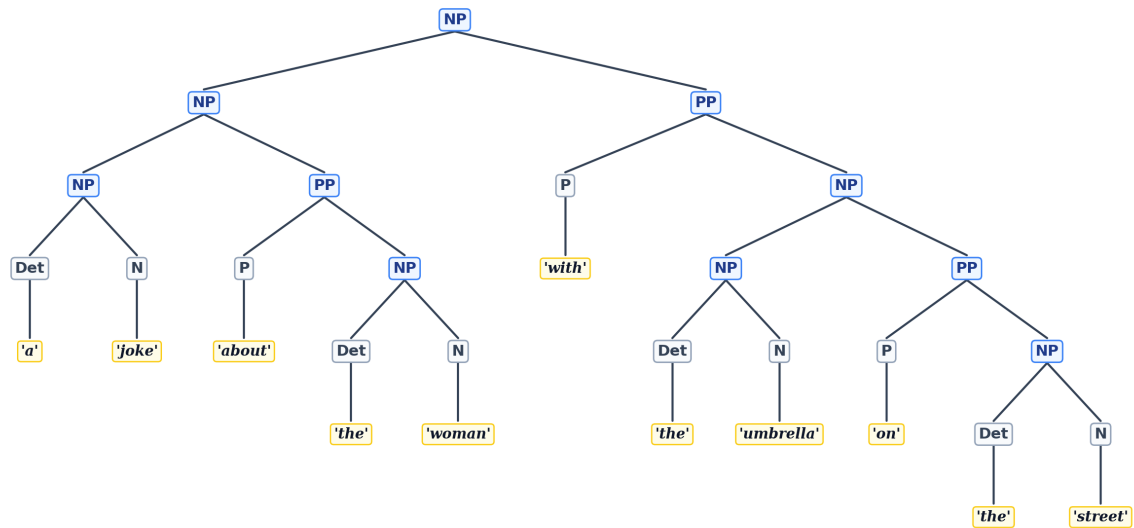


Part A.1 Visualizations: Emitted Parse Trees 4 and 5**Part A — Parse Tree 4 of 5**

PP3 ('on the street') attaches to 'the umbrella', which attaches to 'the woman', which attaches to 'a joke'

**Part A — Parse Tree 5 of 5**

PP1 ('about the woman') attaches to 'a joke'; PP3 ('on the street') attaches to 'the umbrella', which attaches to 'a joke...'



Part B (5 Marks): Structural Ambiguity in 'They are flying planes'

The sentence 'They are flying planes' has two distinct structural interpretations:

1. Progressive Action Reading ('They [pilots] are in the act of flying planes'): 'are' is an auxiliary verb (Aux) combining with the verb phrase [VP [V flying] [NP planes]] via VP → Aux VP.

2. Copular Description Reading ('Those objects are planes that fly'): 'are' is a main/copular verb (V) combining with the modified noun phrase [NP [Adj flying] [N planes]] via VP → V NP.

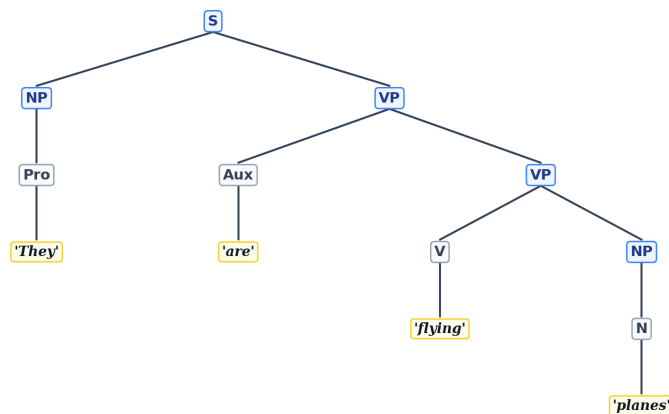
Both required linguistic constraints are respected: (i) NP → Pro allows an NP to be substituted with a pronoun ('They'), and (ii) VP → V NP allows a verb and an immediately following NP to form a VP.

S → NP VP	Pro → 'They' 'they'
VP → Aux VP V NP	Aux → 'are'
NP → Pro N Adj N	V → 'are' 'flying'
Adj → 'flying'	N → 'planes'

Part B Visualizations: Both Parse Trees Emitted by NLTK

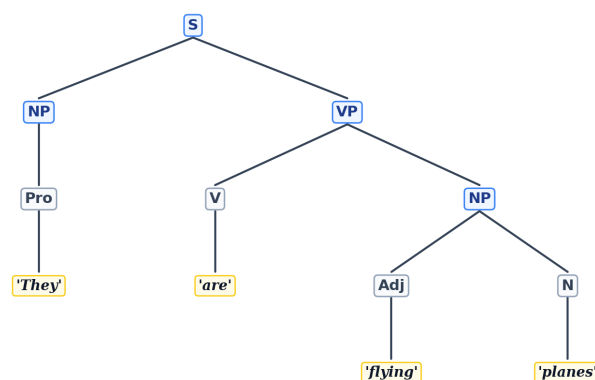
Part B — Parse Tree 1 ('They are flying planes')

Reading 1 (Progressive Action): 'are' is an Auxiliary + [VP [V flying] [NP planes]] ('They are piloting planes')



Part B — Parse Tree 2 ('They are flying planes')

Reading 2 (Copular Description): 'are' is a main/copular Verb + [NP [Adj flying] [N planes]] ('Those objects are planes that fly')



Part C (5 Marks): Extending Grammar for 'Flying planes can be dangerous'

In 'Flying planes can be dangerous', the subject 'Flying planes' is structurally ambiguous:

1. Non-Finite VP Subject ('The activity of flying planes can be dangerous'): 'Flying' (V_nf) and 'planes' (NP) combine via VP_nf → V_nf NP. Following the Hint, we use the unary rewrite rule NP → VP_nf (whose right-hand side is the phrasal category VP_nf) so a non-finite VP can serve as the subject NP of the sentence.

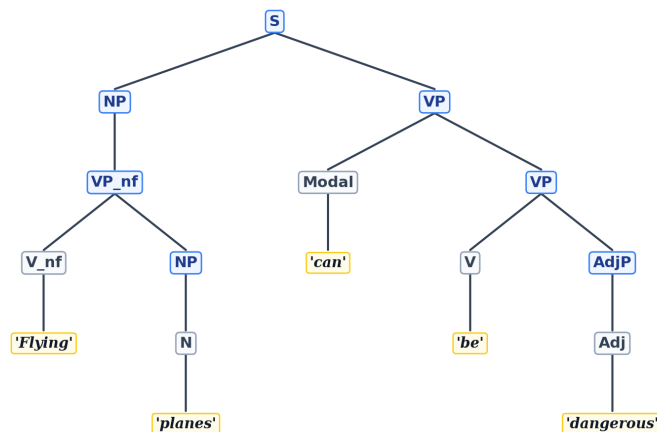
2. Modified NP Subject ('Planes that are flying can be dangerous'): 'Flying' (Adj) modifies 'planes' (N) via NP → Adj N.

S → NP VP	Pro → 'They' 'they'	Aux → 'are'
VP → Aux VP_nf Modal VP V NP V AdjP	Modal → 'can'	V → 'are' 'be'
VP_nf → V_nf NP	V_nf → 'flying' 'Flying'	
NP → Pro N Adj N VP_nf	Adj → 'flying' 'Flying' 'dangerous'	
AdjP → Adj	N → 'planes'	

Part C Visualizations: Both Parse Trees Emitted by NLTK

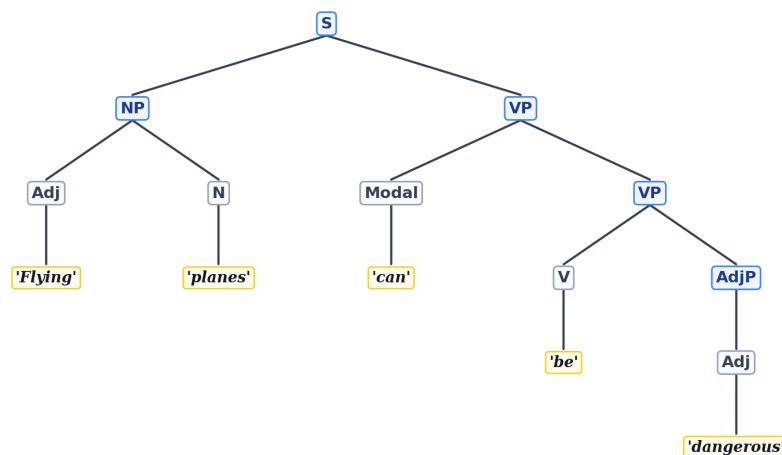
Part C — Parse Tree 1 ('Flying planes can be dangerous')

Reading 1 (Gerundive / Non-Finite VP Subject): [NP [VP_nf [V_nf Flying] [NP planes]]] ('The act of flying planes can be dangerous')



Part C — Parse Tree 2 ('Flying planes can be dangerous')

Reading 2 (Modified Noun Phrase Subject): [NP [Adj Flying] [N planes]] ('Planes that are flying can be dangerous')



Part D (5 Marks): Disambiguation with 'is' vs. 'are'

Why Ambiguity is Eliminated: The modal auxiliary 'can' ('can be') is uninflected for grammatical number, so it accepts both singular and plural subjects. Replacing 'can be' with 'is' or 'are' enforces Subject-Verb Number Agreement:

1. 'Flying planes is dangerous' (Singular Agreement -> 1 Parse): 'is' (V_sg) requires a singular subject (NP_sg). A non-finite VP acting as a subject ('the act of flying planes', NP_sg -> VP_nf) is always grammatically singular (as in 'To err is human'). The plural noun 'planes' is embedded as the object of 'Flying' and is not the head of the subject.

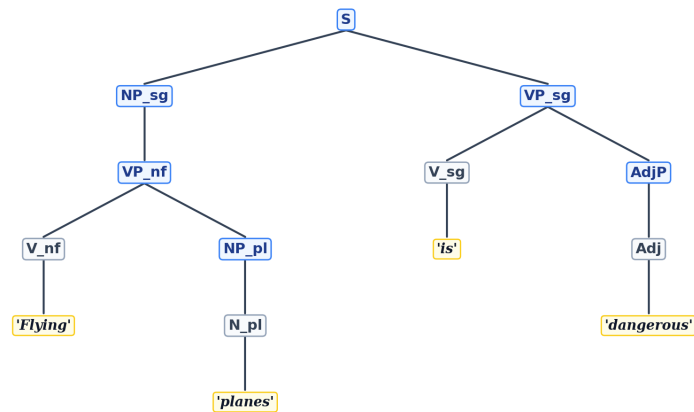
2. 'Flying planes are dangerous' (Plural Agreement -> 1 Parse): 'are' (V_pl) requires a plural subject (NP_pl). When 'Flying' is an adjective modifying the plural head noun 'planes' (NP_pl -> Adj N_pl), the noun phrase is plural and agrees exclusively with 'are'.

```
S -> NP_sg VP_sg | NP_pl VP_pl
NP_sg -> VP_nf
NP_pl -> Pro_pl | N_pl | Adj N_pl
VP_nf -> V_nf NP_pl
VP_inf -> V_inf AdjP
Aux_sg -> 'is'
Aux_pl -> 'are'
VP_sg -> V_sg AdjP | V_sg NP_sg | Aux_sg VP_nf | Modal VP_inf
VP_pl -> V_pl AdjP | V_pl NP_pl | Aux_pl VP_nf | Modal VP_inf
AdjP -> Adj
V_sg -> 'is'
V_pl -> 'are'
V_inf -> 'be'
Modal -> 'can'
V_nf -> 'flying' | 'Flying'
Adj -> 'flying' | 'Flying' | 'dangerous'
N_pl -> 'planes'
```

Part D Visualizations: Disambiguated Parse Trees ('is' vs. 'are')

Part D — 'Flying planes is dangerous' (Disambiguated: Singular Non-Finite VP Subject)

Only 1 Parse: 'is' (V_sg) agrees solely with singular NP_sg -> VP_nf ('The act of flying planes is dangerous')



Part D — 'Flying planes are dangerous' (Disambiguated: Plural Modified NP Subject)

Only 1 Parse: 'are' (V_pl) agrees solely with plural NP_pl -> Adj N_pl ('Planes that fly are dangerous')

